

Assignment 2 | October 2021

Introduction To Machine Learning (ELL784)

Report - Submitted by:

M Rupesh Kumar – 2021EEY7512

Aniruddh Nagar – 2021EET2113

Vivek Noel Soren – 2021EET2122

Contents

1. QUESTION 1: ANN classification task on MNIST dataset	3
1.1. Basic info: Libraries and the dataset	3
1.2. Data pre-processing:	3
1.3. ANN without regularisation	4
1.4. Model training	4
1.5. Output:	5
1.6. Regularization	6
1.6.1. Early stopping	6
1.6.2. ANN model with L2 regularization	7
1.6.3. ANN model with L2 - regularisation and dropout	8
1.7. Conclusion:	9
2. QUESTION 2: CNN classification task with CIFAR-10 dataset	10
2.1. Basic info: Libraries and the dataset	10
2.2. Fixed parameters	10
2.3. Base_line model	11
2.4. Number of Conv layers	12
2.5. Number of Fully - Connected (FC) layers and hidden units	13
2.6. Max-pooling	15
2.7. Number of filters in Convolution layers	16
2.8. Experimenting with Epochs, Strides and Activation functions	17
2.8.1. Epochs	17
2.8.2. Strides	17
2.8.3. Activation Functions	18
2.8.4. Overall model performance summary by tuning above parameters through experiments and trial-error method	18
2.9. Further improving model performance	18
2.9.1. Dropout layers	18
2.9.2. Batch normalisation and Dropout regularisation	19
2.9.3. Miss classified images:	21
2.10. Conclusion	22

1. QUESTION 1: ANN classification task on MNIST dataset

1.1. Basic info: Libraries and the dataset

Libraries: Keras (Built-In Tensorflow)

Dataset: MNIST dataset

```
mnist = tf.keras.datasets.mnist
(x, y), (x_test, y_test) = mnist.load_data()

#normalization
x, x_test = x/255.0, x_test/255.0

x_train, x_val, y_train, y_val = x[:50000], x[50000:], y[:50000], y[50000:]

print("X train = ", x_train.shape)
print("y train = ", y_train.shape)
print("X validation = ", x_val.shape)
print("y validation = ", y_val.shape)
print("X test = ", x_test.shape)
print("y test = ", y_test.shape)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11493376/11490434 [=====] - 0s 0us/step
11501568/11490434 [=====] - 0s 0us/step
X train = (50000, 28, 28)
y train = (50000,)
X validation = (10000, 28, 28)
y validation = (10000,)
X test = (10000, 28, 28)
y test = (10000,)
```

1.2. Data pre-processing:

Out of the 60,000 images of the MNIST dataset, 50,000 images are used as training datasets, while 10,000 images are used as validation data. Another 10,000 images are used for testing the model.

```
print("Shape of images: ", x_train.shape[1:])
print("Shape of labels: ", y_train.shape)
```

```
Shape of images: (28, 28)
Shape of labels: (50000,)
```

```
x_train = x_train.reshape(x_train.shape[0], 28*28)
x_val = x_val.reshape(x_val.shape[0], 28*28)
x_test = x_test.reshape(x_test.shape[0], 28*28)
```

```
print(x_train.shape, x_test.shape, x_val.shape)
```

(50000, 784) (10000, 784) (10000, 784)

The Images were further reshaped into (784) length vectors per sample.

1.3. ANN without regularisation

A neural network model without any regularisation techniques to handle overfitting is constructed with two hidden layers of dimension (500, 1) and an output layer to classify the result into ten different classes. Sparse Categorical Cross Entropy is used as a loss function since the classes are mutually exclusive.

The optimiser used was 'Adam'.

The learning rate is set to 0.0001 (1e-4).

```
# Create model
def get_ANN_model():
    model=Sequential()
    model.add(Input(shape = (28*28,), name = 'Input_layer'))
    model.add(Dense(500, activation='relu', name='hidden_1'))
    model.add(Dense(500, activation='relu', name='hidden_2'))
    model.add(Dense(10, activation = 'softmax', name='Output_layer'))

    model.summary()
    opt=Adam(learning_rate=1e-4)
    model.compile(loss='sparse_categorical_crossentropy',
                  optimizer=opt,
                  metrics=['acc'])

    return model
```

1.4. Model training

The model is trained with a batch size of 64 and an epoch count of 250. Output is given in terms of test accuracy and test loss. A lower test loss value translates to a better model.

```
tf.random.set_seed(42)
dropout_rate = [0.0, 0.2, 0.4]
model = get_ANN_model()
#es = EarlyStopping(monitor = 'val_loss', mode='min', verbose = 1, patience = 5)
ann_model_1 = model.fit(x=x_train,y=y_train,
                        batch_size=64,
                        epochs=250,
                        validation_data=(x_val, y_val),
                        shuffle=False,
                        verbose = 1)

plot_training_params("ANN_1", ann_model_1)
```

Evaluating the Testing Accuracy and loss values :

```
test_loss, test_acc = model.evaluate(x_test, y_test, batch_size = 64);  
print("Test_accuracy: ", test_acc)  
print("Test_loss: ", test_loss)
```

1.5. Output:

The model ran for 250 epochs with the evaluation scores/ results below in the tabular and graphical form.

Table: Model performance after 250 epochs

Epoch 250/250						
metric	acc:	loss:	val_acc:	val_loss:	Test_acc	Test_loss:
Value	1.0000	3.0637e-09	0.9820	0.2095	0.9811	0.1939

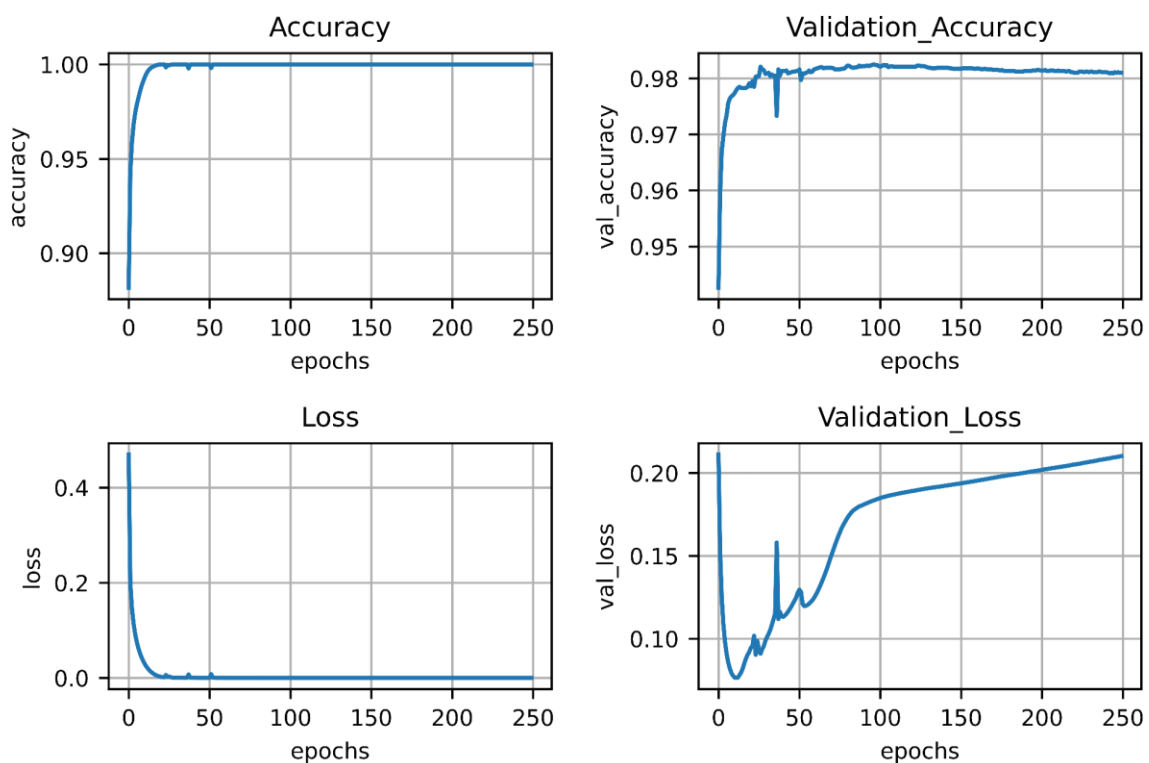


Figure: Simple Feed forward neural network model (a) Training accuracy (b) validation accuracy (c) training loss (d) validation loss

We observe from the plots that the model reaches almost **100% accuracy** just after 20 epochs. On observing the validation-loss curve, **the validation loss seems to increase after 20 epochs, implying the model is OVERFITTING.**

1.6. Regularization

1.6.1. Early stopping

Early stopping is done to stop training when the performance on the validation dataset starts to degrade, precisely when the validation loss seems to increase. It prevents the model from overfitting the training dataset.

```
tf.random.set_seed(42)
dropout_rate = [0.0, 0.2, 0.4]
model = get_ANN_model()
es = EarlyStopping(monitor = 'val_loss', mode='min', verbose = 1, patience = 5)
ann_model_1 = model.fit(x=x_train,y=y_train,
                        batch_size=64,
                        epochs=250,
                        validation_data=(x_val, y_val),
                        shuffle=False,
                        callbacks = [es],
                        verbose = 1)

plot_training_params("ANN_1_early_stopping", ann_model_1)
```

In our model, the training stops at the 17th epoch to prevent any further degradation.

Table: Model performance after introducing Early stopping

Epoch 17/250 -----> Epoch 00017: early stopping						
Metric	acc:	loss:	val_acc:	val_loss:	Test_acc:	Test_loss:
Value	0.9991	0.0073	0.9781	0.0854	0.9774	0.0823

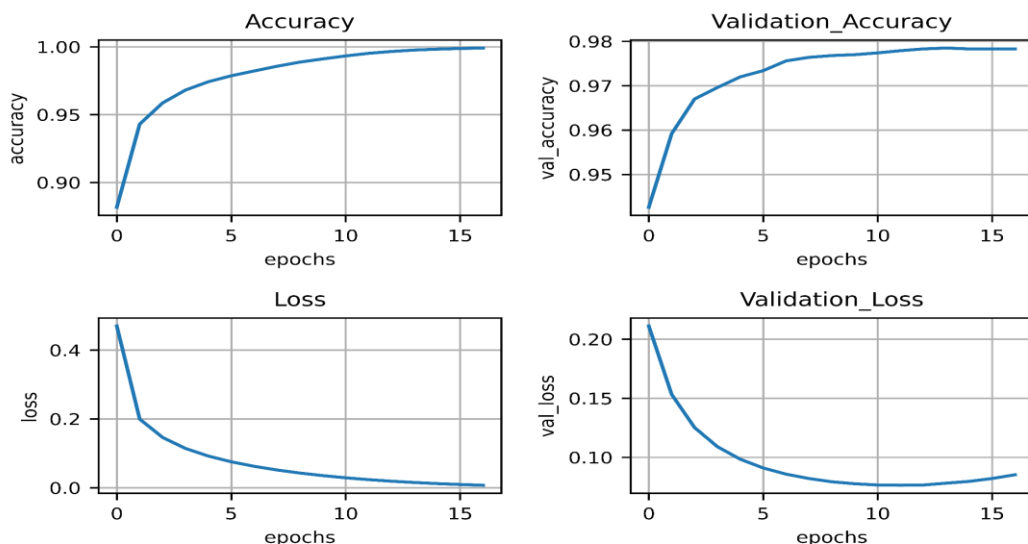


Figure: ANN model with Early stopping model (a)Training accuracy (b)validation accuracy (c)training loss (d) validation loss

Early stopping monitors the validation loss and stops the training process when the validation loss continuously increases for **five** consecutive epochs.

1.6.2. ANN model with L2 regularization

```
# Create model
def get_ANN_dropout_model():
    model=Sequential()
    model.add(Input(shape = (28*28), name = 'Input_layer'))
    model.add(Dense(500, activation='relu', name='hidden_1', kernel_regularizer='l2'))
    model.add(Dropout(0.4))
    model.add(Dense(500, activation='relu', name='hidden_2', kernel_regularizer='l2'))
    model.add(Dropout(0.2))
    model.add(Dense(10, activation = 'softmax', name='Output_layer'))

    model.summary()
    opt=Adam(learning_rate=1e-4)
    model.compile(loss='sparse_categorical_crossentropy',
                  optimizer=opt,
                  metrics=['acc'])

    return model
```

```
tf.random.set_seed(42)

model = get_ANN_L2_reg_model()
es = EarlyStopping(monitor = 'val_loss', mode='min', verbose = 1, patience = 5)
ann_model_1 = model.fit(x=x_train,y=y_train,
                        batch_size=64,
                        epochs=250,
                        validation_data=(x_val, y_val),
                        shuffle=False,
                        callbacks = [es],
                        verbose = 1)

plot_training_params("ANN_L2_reg", ann_model_1)
```

```
test_loss, test_acc = model.evaluate(x_test, y_test,batch_size = 64);
print("Test_accuracy: ", test_acc)
print("Test_loss: ", test_loss)
```

Table: Model performance after introducing L2 regularisation

Epoch 250/250						
Metric	acc:	loss:	val_acc:	val_loss:	Test_acc	Test_loss:
value	0.9883	0.1287	0.9777	0.1556	0.9779	0.1547

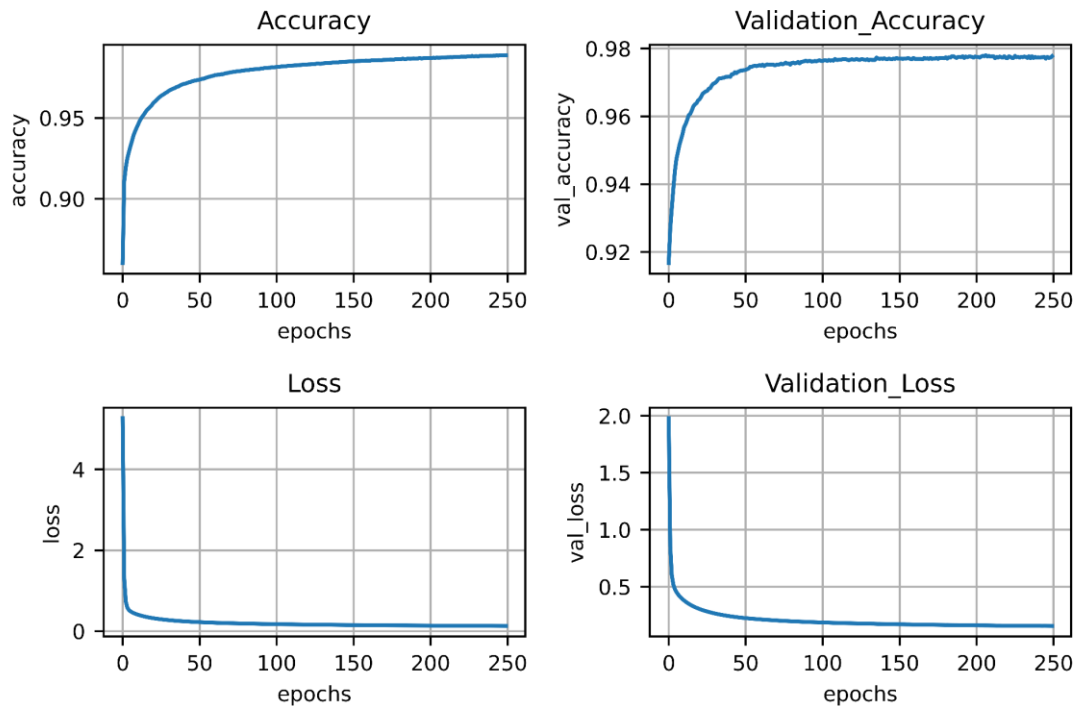


Figure: ANN model with L2 regularization model (a) Training accuracy (b) validation accuracy (c) training loss (d) validation loss

From observing the model performance from the metrics plots and model performance table, we see that after introducing the L2 regularisation, the model learning curve is gradual and smooth. We can observe a slight marginal performance improvement from 98.16% to 98.83%.

1.6.3. ANN model with L2 - regularisation and dropout

Model configuration with regularisation and dropout are as follows.

```
# Create model
def get_ANN_dropout_model():
    model=Sequential()
    model.add(Input(shape = (28*28), name = 'Input_layer'))
    model.add(Dense(500, activation='relu', name='hidden_1', kernel_regularizer='l2'))
    model.add(Dropout(0.4))
    model.add(Dense(500, activation='relu', name='hidden_2', kernel_regularizer='l2'))
    model.add(Dropout(0.2))
    model.add(Dense(10, activation = 'softmax', name='Output_layer'))

    model.summary()
    opt=Adam(learning_rate=1e-4)
    model.compile(loss='sparse_categorical_crossentropy',
                  optimizer=opt,
                  metrics=['acc'])
    return model
```


Table: Model performance after introducing Dropout and L2 Regularisation

Epoch 96/250 -----> Epoch 00096: early stopping						
Metric	acc:	loss:	val_acc:	val_loss:	Test_acc	Test_loss:
Value	0.9720	0.2185	0.9777	0.1556	0.9720	0.2185

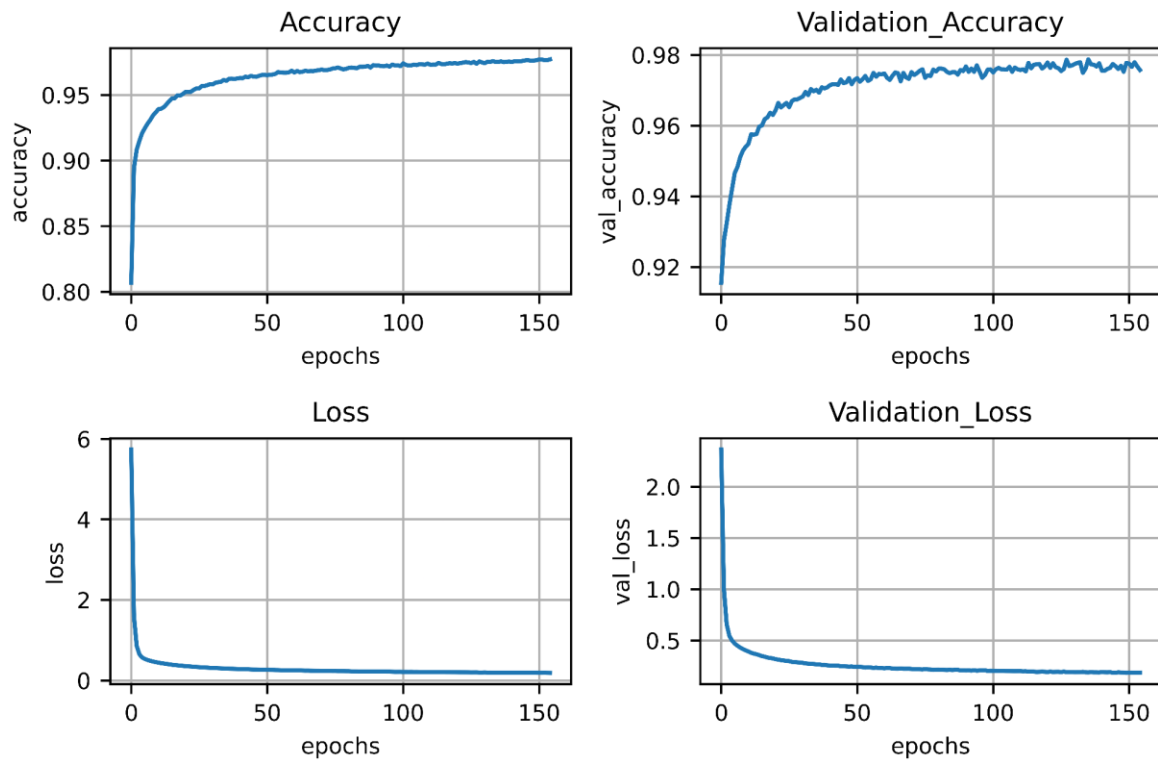


Figure: ANN model with Dropout and L2 regularisation model (a) Training accuracy (b) validation accuracy (c) training loss (d) validation loss

1.7. Conclusion:

- The model's (without regularization) validation accuracy fluctuates with increase in epochs, the validation loss increases after about 20 epochs, implying overfitting.
- There is marginal improvement in overall accuracy after introduction of the dropout layers.
- Despite adding the dropout regularisation, we see no significant improvement in the model classification accuracy, but on observing the validation loss curves, we see that the model generalizes well to the validation data and doesn't overfit the training data.

2. QUESTION 2: CNN classification task with CIFAR-10 dataset

2.1. Basic info: Libraries and the dataset

Libraries: Keras (built-in tensorflow)

Dataset: CIFAR-10 dataset

- `X_train = (50000, 32, 32, 3)` #Training data
- `y_train = (50000, 1)` #Training data class labels
- `X_test = (10000, 32, 32, 3)` #Testing Data
- `y_test = (10000, 1)` #testing data class labels

Table: classes of image and used class label indexing

<i>Class</i>	<i>Class label</i>	<i>Class</i>	<i>Class label</i>
0	aeroplane	5	dog
1	automobile	6	frog
2	bird	7	horse
3	cat	8	ship
4	deer	9	truck

NOTE: 10% of the training data was used as the validation set (for experiments 1-16), i.e. **45,000 images for training, 5,000 images for validation and 10,000 images for testing.**

2.2. Fixed parameters

Throughout the entire experiments, the following parameters were maintained as constants:

Parameters	Value used
Loss	Sparse Categorical Cross entropy
Optimizer	Adam
Learning rate	1e-4
Batch size	64
Early Stopping	Patience = 10 epochs Monitoring: Validation loss ('min val_loss')

NOTE:

The final output layer contains **ten hidden_units**, and the activation function used was the "**SoftMax**" function.

Early stopping with patience of 10 was used throughout all the experiments, monitoring the validation loss.

2.3. Base_line model

As a thumb rule,

The number of filters used was powers of 2 (i.e. 16, 32, 64, 128 etc.).

The number of units used in Fully-Connected (FC) layers is in powers of 2(i.e. 16, 32, 64, 128 etc.).

Kernel sizes used were (3,3).

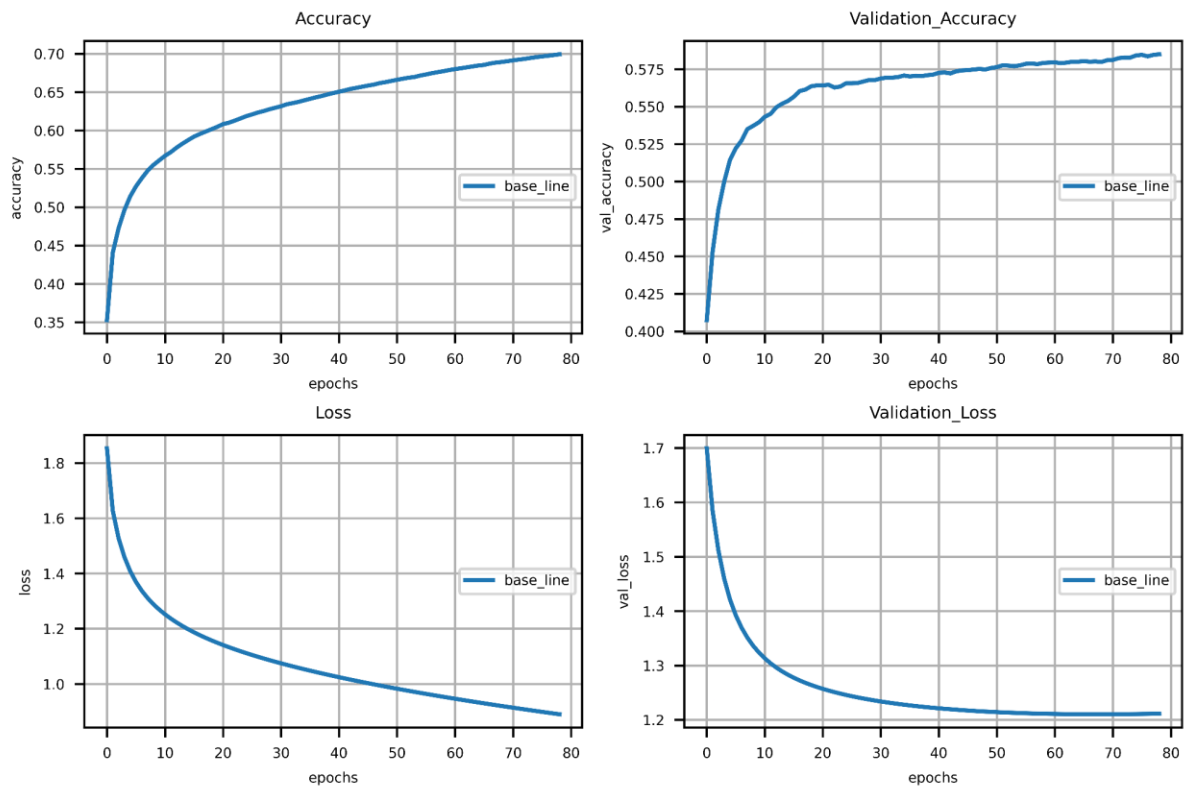


Figure: Base_line model (a)Training accuracy (b)validation accuracy (c)training loss (d) validation loss

We create a model with 1 CNN layer and a final output layer. The result of this particular model is kept as the baseline, and the model parameters are modified to improve upon this result. The model summary is given below.

Layer (type)	Output Shape	Param #
CNN_layer_0 (Conv2D)	(None, 30, 30, 16)	448
flatten_5 (Flatten)	(None, 14400)	0
Output_layer (Dense)	(None, 10)	144010
Total params: 144,458		
Trainable params: 144,458		
Non-trainable params: 0		

The model results observed are as follows.

Table: BASE_LINE Model result summary

Parameter	Value observed
Training accuracy	0.6984
Training loss	0.8932
Validation accuracy	0.5848
Validation loss	1.211
Testing Accuracy	0.5697
Testing loss	1.2321

2.4. Number of Conv layers

Five experiments were conducted by increasing the number of "Conv" layers from 1 to 5 layers keeping the number of filters 16 or 32. Also, an additional "Fully-Connected" layer with 64 units is introduced after the "flatten" layer. Max-pool layers were NOT used in this experiment. The stride length was maintained to be (1,1) throughout all the layers.

Fixed parameters:

- Number of filters = 16 (or) 32, each of size = (3,3), stride_length = (1,1)
- FC layers =1 => 64 units
- Activation function = 'ReLU'

Table: Analysis of model performance with the number of Conv layers.

Experiment	Number of conv layers	Filter size	Training accuracy	Validation accuracy	Test Accuracy	Trainable parameters
1	1	16	0.74235	0.58899	0.57690	1,049,738
2	2	16,16	0.77793	0.62680	0.6130	1,052,058
3	3	16,16,32	0.77619	0.62139	0.6194	2,105,274
4	4	16,16,32,32	0.76253	0.61500	0.6266	2,116,650
5	5	16,16,32,32, 64	0.80391	0.62599	0.6297	4,230,170

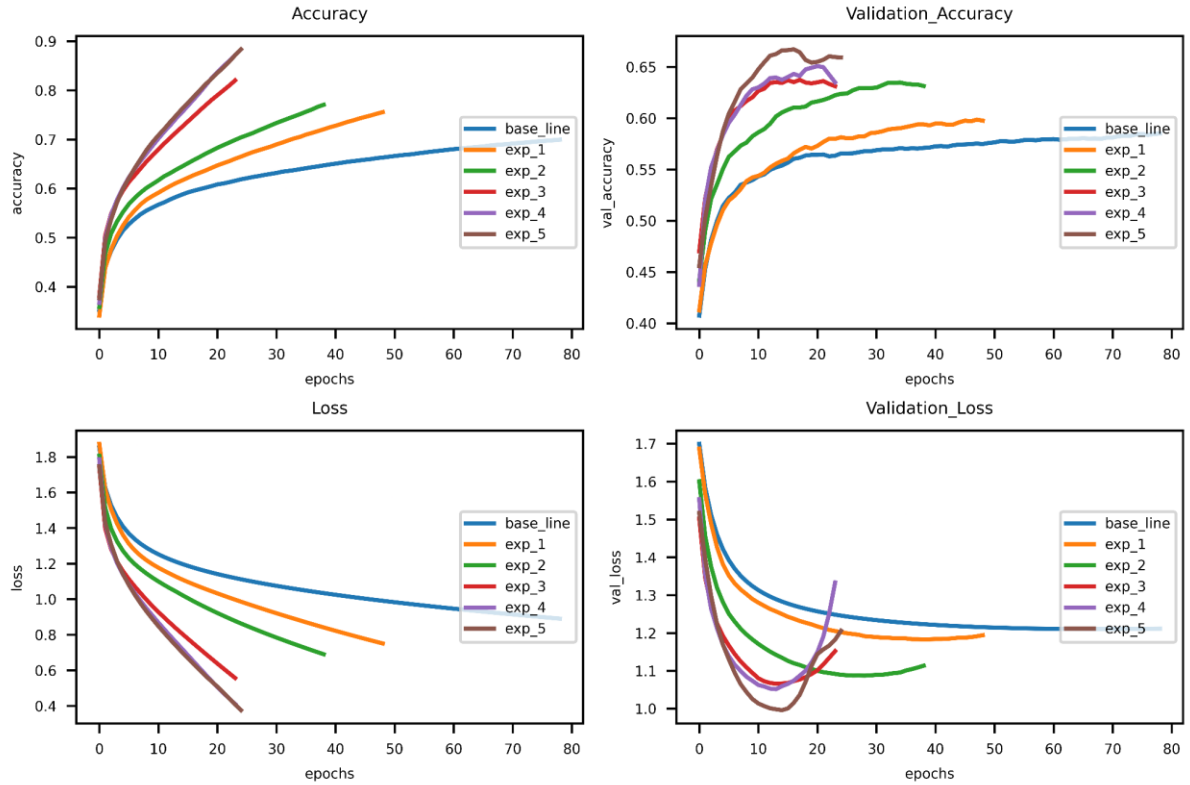


Figure: Base_line model (a)Training accuracy (b)validation accuracy (c)training loss (d) validation loss

The validation accuracy increases with the increase in the number of Conv layers and poses a risk of overfitting the training data. On observing the table, there is no significant improvement in the validation accuracy after utilising more than 4 CNN layers.

So, the next set of experiments are conducted utilising a fixed set of **4 CNN layers**.

2.5. Number of Fully - Connected (FC) layers and hidden units

The following experiments were conducted to identify the number of "Fully-connected" layers after the flatten and before output layer. The following three experiments were conducted.

Fixed parameters:

- 4 Conv layers, Number of filters = (16,16,32,32) respectively, kernel_size = (3,3), stride_length = (1,1), activation_function = 'ReLU'
- Activation function = 'ReLU'

Table: Analysis of model performance with number of FC layers.

Experiment	No. of FC layers	Hidden units	Training accuracy	Validation accuracy	Test Accuracy	Trainable parameters
4	1	64	0.81955	0.63080	0.61760	2,116,650
6	2	64,32	0.74260	0.69099	0.67320	2,116,282
7	3	64,32,16	0.78208	0.67739	0.67000	2,116,650

The above table shows that the **2 FC layers** with **64 and 32 units**, respectively, yield the highest test accuracy.

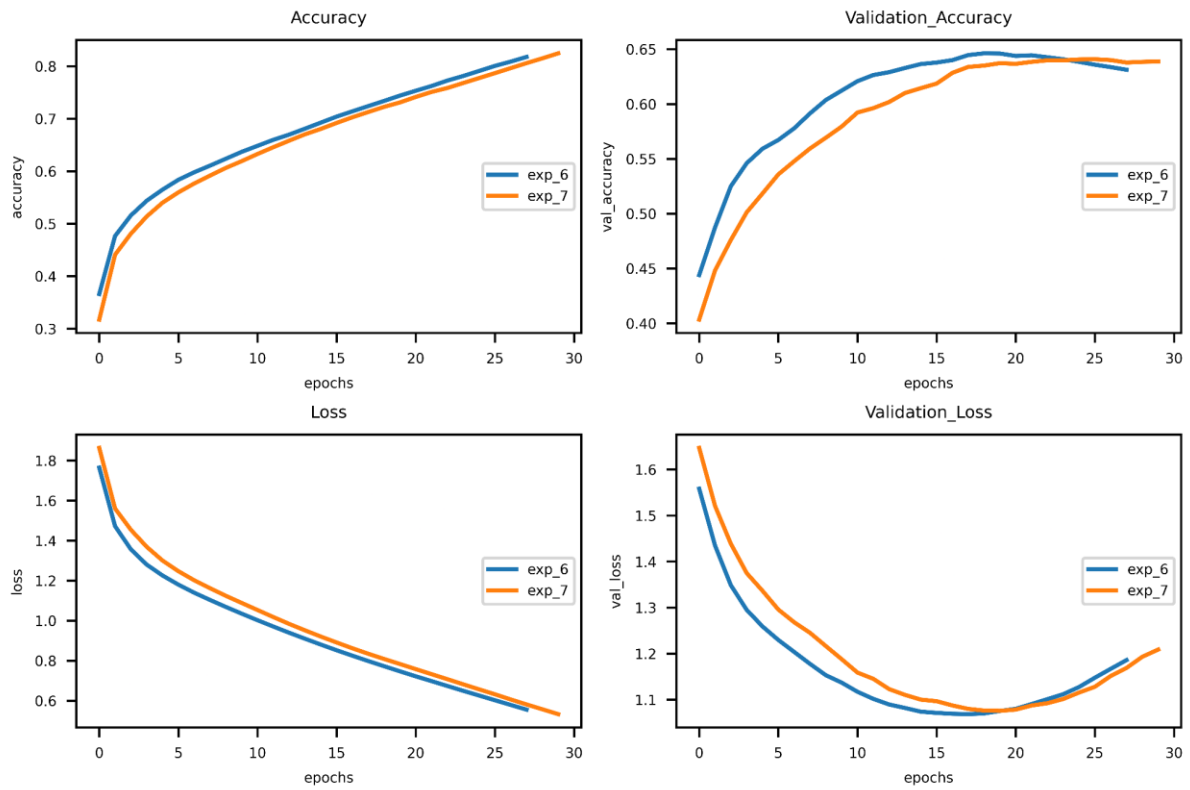


Figure: Experiment 6-7 model (a) Training accuracy (b) validation accuracy (c) training loss (d) validation loss

2.6. Max-pooling

Utilising the previous configuration with 4 Conv layers, max-pool layers were introduced after each convolutional layer and model performances were analysed.

Table: Different configurations of max-pool layers

Experiment	Max-pool layers inserted after=>				---			
	After Conv_1	After Conv_2	After Conv_3	After Conv_4	Train accuracy	Validation Accuracy	Test Accuracy	Trainable parameters
8	Y	Y	Y	Y	0.78208	0.6774	0.6669	27,690
9	-	Y	-	Y	0.79804	0.7110	0.6682	150,570
10	-	Y	Y	-	0.8014	0.7129	0.6898	150,570

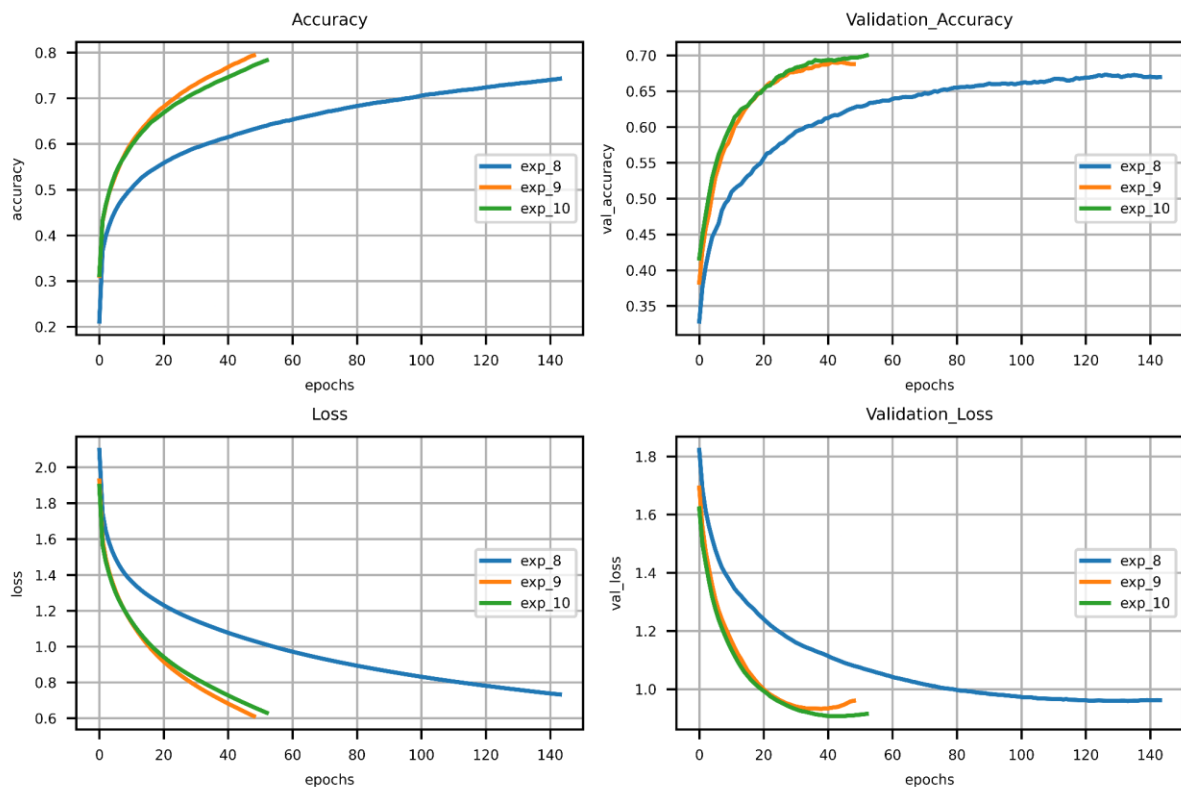


Figure: Experiment 8-10 model (a) Training accuracy (b) validation accuracy (c) training loss (d) validation loss

On observing the above table, we can identify that the max-pool layers after the **2nd and the 3rd Conv layers** yield relatively superior results. The rest of the experiments will be conducted with this configuration.

2.7. Number of filters in Convolution layers

These sets of experiments now revolve around finding the optimal number of filters in the *Conv* layers.

Fixed parameters:

- 2 FC layers with 64 and 32 units respectively, *activation_function* = 'ReLU'
- 4 CNN layers, max-pool layer after layers two and layer_3

Table: Number filters used in each convolutional layer and the model performance.

Experiment	Number of filters in Conv layers				Train accuracy	Validation Accuracy	Test Accuracy	Trainable parameters
	Conv_1	Conv_2	Conv_3	Conv_4				
11	16	32	64	128	0.7190	0.6298	0.6213	133,050
12	16	16	64	128	0.8239	0.7160	0.6941	613,034
13	32	32	64	128	0.860	0.7175	0.7035	629,626
14	16	16	32	64	0.7983	0.6901	0.6838	290,890
15	16	32	32	64	0.8353	0.7175	0.7031	297,818
16	32	64	128	256	0.9067	0.7315	0.7113	1,439,834

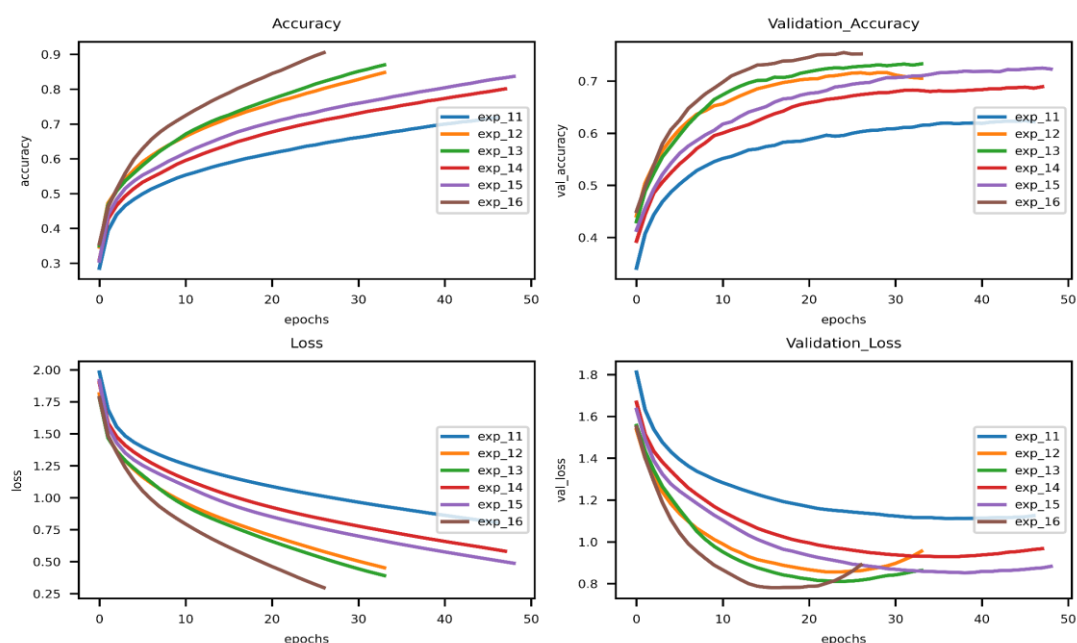


Figure: Experiment 11 -16: model (a)Training accuracy (b)validation accuracy (c)training loss (d) validation loss

On experimenting with the various number of filters in each of the layers, we observe that the number of trainable parameters increases, implying that the training ability of the model improves significantly with the increased number of filters. On observing, **32, 64, 128, 256 (Experiment 16)** configuration yields the best results. For the rest of the experiments, the filter configuration will be maintained.

2.8. Experimenting with Epochs, Strides and Activation functions

After this, the model was trained for the training set, and the validation set was entirely used for training the model.

2.8.1. Epochs

The number of epochs at which model training was stopped varies with the learning rate specified and the number of trainable parameters within the system. In order to get a generalised performance and avoid overfitting, the "Early stopping" was used, with the patience of 15 epochs, monitoring the validation loss.

Table: Metrics observed after a specific set of epochs

Epochs	Training accuracy	Training loss	Testing Accuracy	Testing loss
25	0.9072	0.2934	0.7318	0.9668
50	0.9863	0.0427	0.7245	1.9803
75	0.9940	0.0175	0.7334	2.4567
100	0.9964	0.0116	0.7357	2.6510
150	0.0079	0.9975	0.7366	2.9115

We are interested in finding the model with a high-test accuracy and the lowest test loss possible. On observing the data over epochs, we see that after 50 epochs, there is no improvement in Testing accuracy, but the testing loss continuously increases. This observation implies that the model tends to **overfit** the training data after **about 50 epochs**, and thus the model training should be **stopped after 50 epochs**.

2.8.2. Strides

The number of trainable parameters is significantly reduced with an increased stride length of convolutional layers or the MaxPooling layers. However, the model performance has not improved. The number of trainable parameters is significantly reduced, reducing the training time significantly. The strides performance is similar to that of the max-pooling layer. However, we still use the max-pooling layers between the CNN layers.

2.8.3. Activation Functions

Three activation functions were experimented with, namely 'ReLU' and 'LeakyReLU'.

Of the three, ReLU activation yields the best results. However, there is no significant difference in performance models when either the 'LeakyReLU' or 'ReLU' model is used.

Table: Metrics observed with various activation functions

Activation function	Training accuracy	Training loss	Testing Accuracy	Testing loss
'Tanh'	0.9960	0.0129	0.7041	4.1972
'ReLU'	0.9772	0.2934	0.7318	0.9668
'LeakyRelu'	0.9863	0.0427	0.7345	0.9803

2.8.4. Overall model performance summary by tuning above parameters through experiments and trial-error method

Table: Final Model (without dropout or regularisation) result summary

Metric	Training accuracy	Training loss	Testing Accuracy	Testing loss
value observed	0.9772	0.2934	0.7355	0.9668

We observe an **73.55% classification accuracy** on test set without any regularization.

2.9. Further improving model performance

2.9.1. Dropout layers

Dropout layer was introduced after every max_pooling layer and Dense layers (Hidden layers).

This time the entire training dataset was used for the Training purpose, and the final model after 150 epochs was used for model evaluation.

Table: Model after introducing dropout layers for regularisation

Metric	Training accuracy	Training loss	Testing Accuracy	Testing loss
value observed	0.8721	0.3956	0.7972	0.7224

The dropout regularisation proves to be effective and has increased the classification accuracy from 73.55% to almost **80%**. Batch normalisation is introduced and discussed in the next section to improve model performance on the classification task further.

2.9.2. Batch normalisation and Dropout regularisation

A batch normalisation layer is introduced after every convolutional layer and before applying the activation function.

Table: Model metrics after adding dropout and batch normalisation

Metric	Training accuracy	Training loss	Testing Accuracy	Testing loss
value observed	0.9306	0.2025	0.8200	0.78983

The best **TEST accuracy** observed was **82.00%** with the following model configuration.

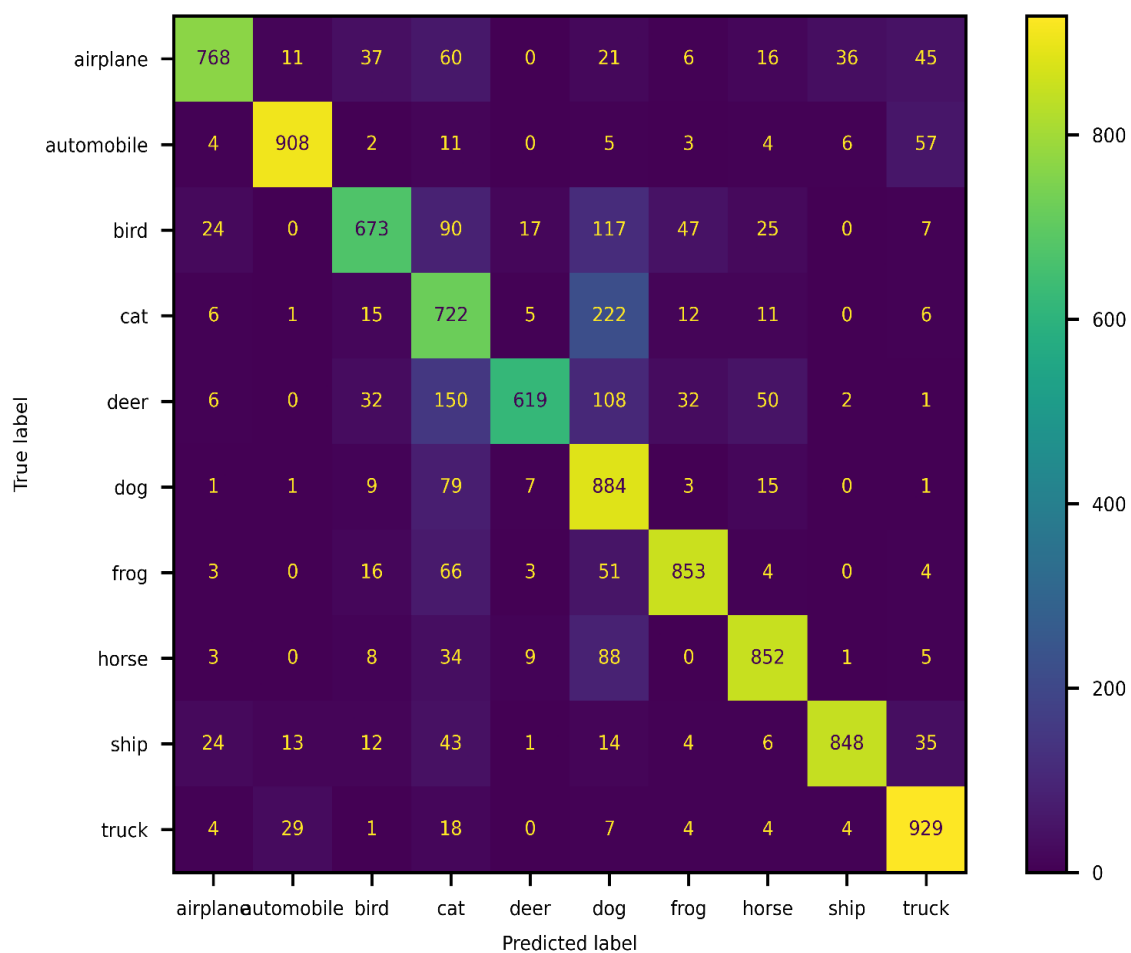


Figure: Confusion matrix

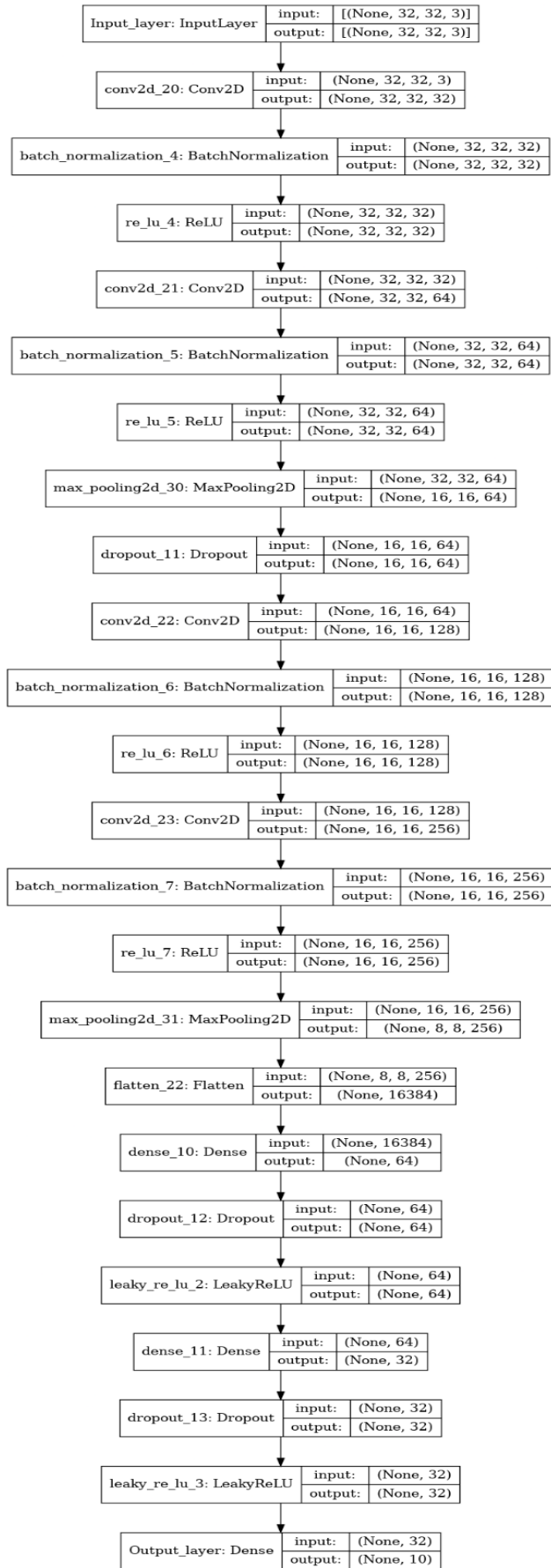
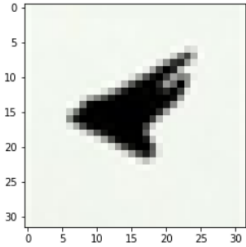
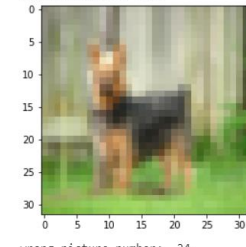
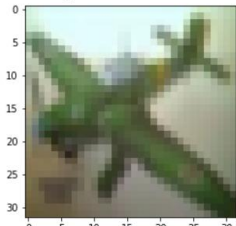
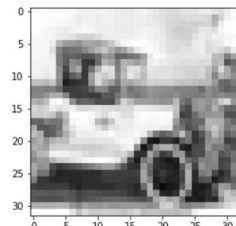
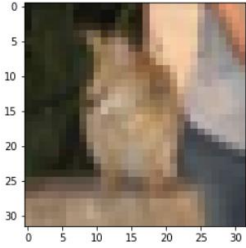


Figure: The final model configuration

2.9.3. Miss classified images:

Table: Misclassified images and possible reasons for misclassification

Image	True Label	Predicted label	The possible reason for misclassification
 wrong_picture_number: 21 TRUE_CLASS: airplane PREDICTED_CLASS: bird	aeroplane	bird	It is tough to identify if it is a bird or plane, even by visual introspection. It is tolerable when the CNN model misclassifies this image.
 wrong_picture_number: 24 TRUE_CLASS: dog PREDICTED_CLASS: deer	dog	deer	The colour and structure of the dog and the deer is similar, which could have led to the misclassification.
 PREDICTED_CLASS: deer wrong_picture_number: 27 TRUE_CLASS: airplane PREDICTED_CLASS: bird	aeroplane	bird	The shape of the aeroplane is similar to that of a bird, which could be the reason for the misclassification.
 wrong_picture_number: 37 TRUE_CLASS: automobile PREDICTED_CLASS: truck	automobile	truck	The structure of this automobile and a truck is similar, which might be the reason for the misclassification.
 wrong_picture_number: 46 TRUE_CLASS: cat PREDICTED_CLASS: frog	cat	frog	The dog's outline is close to that of a frog, which could have caused difficulty in the classification task.

2.10. Conclusion

- The model performance tends to increase with the number of Convolutional layers used but also poses a risk of overfitting the training data. Early hidden Conv layers extract basic features like edges, lines etc., and the deeper hidden Conv layers extract more sophisticated features.
 - The number of fully connected layers and their hidden units significantly influence the classification performance.
 - Max pooling layers are used to subsample and allow only the most significant feature information to propagate to the next set of layers. We need to adjust the number of max-pooling layers to propagate optimal and more compact features to the next layers.
 - Initially, the model fits the training data appropriately and generalises well to test data that is previously not seen by the model. However, as after certain epochs, we observe that the model training accuracy improves significantly, almost reaching 100%, but the test accuracy is still at **approx—74%**, implying that the model started overfitting the training data. Thus, we need to stop training at an optimal number of epochs.
 - The 'ReLU' and the 'Leaky ReLU' activation function perform well in almost all configurations when experimenting with different activation functions.
 - Dropout and Batch-normalisation were introduced to enhance the model performance further and avoid overfitting the training data, which significantly improved test accuracy to **82%**.
-